

RepoTracer 1.0: Ownership-First Repository Retrieval for Coding Agents

Architecture, Activity-Based Failure Control, and Paired Empirical Evaluation

RepoTracer Contributors

30 August 2026

Abstract

RepoTracer delegates broad, read-only repository investigation from a primary coding agent to a smaller Scout model, then returns a bounded evidence map with validated source citations. This report describes the version 1.0 architecture, the ownership-first routing policy, an activity-based process timeout, and the empirical evidence used for the release decision. In a focused three-task paired evaluation, RepoTracer and the direct baseline each solved two of three tasks exactly and each passed 147 of 151 feature checks. RepoTracer passed 1,956 of 1,956 regression checks compared with 1,952 for the baseline, while complete measured provider cost fell by 27.71%. Aggregate wall time increased by 16.83%, so the evidence does not support a latency improvement. A separate 72-trial blind study selected medium Scout reasoning: it achieved 24 of 24 perfect grades; high reasoning produced no quality gain and took 2.38 times the arm-level median wall time. Five repeated process fixtures showed that active runs survived beyond a 200 ms idle interval in all candidate trials, while silent children and descendants were terminated in all trials. These results support the design as a release candidate, not a repository-wide effect estimate. The report follows the ACM SIGSOFT standards for engineering research and software-system benchmarking where the available evidence permits, and states each material deviation.

Contents

Report status	2
1. Objective and research questions	2
2. Contributions	3
3. System design	3
3.1 Request path	3
3.2 Ownership-first routing	4
3.3 Isolated read-only Scout	4
3.4 Citation contract	4
3.5 Activity-based process timeout	5
4. Empirical method	5
4.1 Standards used	5
4.2 Experimental units and treatments	6
4.3 Quality measures	6
4.4 Cost and token accounting	6
4.5 Routing studies	7
4.6 Scout reasoning study	7

4.7 Timeout fixture	7
4.8 Execution environment	7
5. Results	8
5.1 RQ1: end-to-end quality, cost, and latency	8
5.2 Supporting task evidence	8
5.3 RQ2: routing quality	9
5.4 RQ3: Scout reasoning level	9
5.5 RQ4: inactivity timeout and process-tree cleanup	9
6. Interpretation	10
7. Threats to validity	10
7.1 Construct validity	10
7.2 Internal validity	11
7.3 External validity	11
7.4 Conclusion validity	11
8. Reproducibility and artifact record	11
9. Standards conformance assessment	12
9.1 ACM SIGSOFT Benchmarking standard	12
9.2 ACM SIGSOFT Engineering Research standard	13
9.3 Reproducibility and responsible-reporting checklist	13
10. Security, ethics, and broader impacts	14
11. Conclusions	14
References	14

Report status

Document identifier: TR-2026-01

Software version: RepoTracer 1.0.0

Source baseline used by the final preregistered protocol:

603b196d444969dbdaabbbe2852cebe0cc8136de

Evidence period: 27–30 August 2026

Review status: Internal technical report; not peer reviewed and not independently reproduced

This report is an engineering and benchmarking record. It does not claim an ACM artifact badge, statistical significance for the three-task final comparison, or generality beyond the measured workloads. Sanitised protocols, aggregate results, benchmark runners, task definitions, and checksums are retained in the repository. Credential-bearing provider traces and private model sessions are not published.

1. Objective and research questions

Repository-scale coding work has two competing costs. A primary model must gather enough context to change the correct ownership surface, but broad searching consumes expensive model turns and can crowd implementation evidence out of the context window. Delegating every lookup is also wrong: a second model adds latency and can over-route a local task.

RepoTracer addresses the narrower problem: delegate broad repository investigation only when the requested change crosses ownership boundaries or requires an exhaustive inventory, then return a small, validated evidence map to the primary agent. The primary agent retains responsibility for edits and verification.

The evaluation asks four questions.

- **RQ1, end-to-end effect.** Under paired repository tasks, does RepoTracer preserve functional quality while changing complete provider cost and wall time?
- **RQ2, routing.** Can an ownership-first policy select Scout for broad work and stay local for a bounded change, including cases where the source path is initially unknown?
- **RQ3, Scout reasoning.** Which supported reasoning level preserves repository-answer quality at the lowest measured latency and equivalent usage cost?
- **RQ4, failure control.** Can a Scout run longer than its idle interval while meaningful stream activity continues, and does the runtime terminate a silent child together with its descendants?

The primary release criterion is functional quality. Cost is secondary. Latency, routing, citation validity, and process cleanup are independent constraints rather than substitutes for correctness.

2. Contributions

This work makes four engineering contributions.

1. **Ownership-first routing.** The integration distinguishes a local ownership surface from an exhaustive or cross-owner investigation. Unknown file location alone does not force delegation.
2. **Read-only evidence handoff.** A Scout operates in an isolated Codex app-server thread with bounded Read, Glob, and Grep access. It returns a concise finding and at most five repository citations. RepoTracer canonicalises and validates every cited path and line range before handoff.
3. **Activity-based failure control.** The subscription backend uses a resettable Tokio deadline. Valid JSON stream frames and non-whitespace stderr output reset the idle deadline; process existence does not. Silence terminates the child process group and waits for cleanup.
4. **Paired evaluation with complete accounting.** Treatment and baseline runs use the same task, repository commit, prompt, parent model, reasoning setting, and execution limits. Reported treatment cost includes both the primary model and Scout.

3. System design

3.1 Request path

The production request path is:

```
primary Codex model
-> MCP repo_scout(query)
  -> RepoTracer MCP server
    -> isolated codex app-server process
      -> Scout model with read-only repository tools
    <- structured answer and source locations
  <- canonicalised, validated citations and bounded excerpts
-> primary model edits and verifies the repository
```

The default release configuration uses GPT-5.6 Sol as the primary coding model and GPT-5.6 Luna with medium reasoning as Scout. The provider executes through the user's existing Codex subscription and `codex app-server`; RepoTracer does not hold a separate provider credential.

The Rust workspace separates repository tools, model access, Scout policy, MCP transport, CLI integration, and benchmark execution. The principal components are:

Component	Responsibility
<code>repotracer-repo-tools</code>	Root-bounded Read, Glob, Grep, and concurrent tool execution
<code>repotracer-model</code>	OpenAI-compatible model client and mock backend
<code>repotracer-core</code>	Scout loop, limits, configuration, and citation parsing
<code>repotracer-mcp</code>	MCP stdio server and <code>repo_scout</code> contract
<code>repotracer</code>	CLI, Codex setup, subscription backend, doctor, and server
<code>repotracer-bench</code>	Paired manifests, execution plans, and result collection

3.2 Ownership-first routing

Routing begins with the ownership surface requested by the task, not with the number of unknown paths. A request about one command, function, symbol, file, or local behaviour starts with one targeted lookup. A request for every caller, an exported API blast radius, a dependency map, an exhaustive test inventory, or propagation across multiple owners calls Scout first. If the targeted local lookup reveals multiple owners or fails to localise the work, the policy escalates to Scout.

This boundary corrects a rejected policy that treated pathless local work as broad by default. That earlier version recovered broad routing misses but sent a local JSON-version task to Scout in all three repeats. Version 2.1 added contrasting local and Scout examples and made ownership explicit.

The installed policy is a managed block in the Codex instruction file. Setup also registers the MCP server. Uninstall removes both managed additions while preserving unrelated user configuration.

3.3 Isolated read-only Scout

Each subscription Scout call creates an ephemeral Codex home with mode 0700 on Unix. It makes the active provider authentication and selected provider configuration available to the child, but excludes personal instructions, inherited MCP servers, hooks, plugins, applications, browser tools, image tools, and multi-agent tools. The child receives read-only permissions, no approval path, no project instruction bytes, the repository root, and a structured output schema.

This isolation has two purposes. First, the Scout cannot recursively invoke RepoTracer or inherit unrelated tools. Second, repository evidence comes from a small, auditable capability set. The parent agent remains the sole writer.

3.4 Citation contract

Scout output contains an answer plus zero to five citations. Each citation names a repository-relative path, a start line, an end line, and a reason. Validation rejects:

- line zero, reversed ranges, and starts beyond end-of-file;
- paths that cannot be canonicalised;
- paths outside the canonical repository root, including symlink escapes;
- directories and missing files;
- binary files containing NUL bytes.

Valid end lines are clamped to the file’s actual line count. Returned paths use repository-relative forward-slash form. This proves that a citation resolves to a readable source range; it does not prove that the cited range logically entails the Scout’s claim. Logical relevance is evaluated separately by task checks or blind review.

3.5 Activity-based process timeout

A fixed wall-clock cap confuses duration with failure. A healthy model can run longer than a nominal timeout while streaming tool calls, token updates, or answer fragments. Conversely, a child process can remain alive indefinitely without making progress.

Let τ be the configured idle interval and let a_k be the time of the most recent meaningful activity event. The deadline after event k is

$$D_k = a_k + \tau.$$

The runtime terminates the process tree at time t only if

$$t \geq D_k$$

and no later valid activity event has arrived. Therefore, total run time is unbounded by τ when every silent gap is shorter than τ .

The implementation uses one resettable `tokio::time::Sleep` inside `tokio::select!`. A successfully parsed JSON line from stdout sends an activity signal. Stderr sends activity only when a read contains at least one non-whitespace byte. Empty reads, whitespace-only stderr, and process existence do not reset the deadline. The activity channel has capacity one because multiple events before the loop consumes a signal have the same state transition: move the deadline to the current time plus τ .

On expiry, Unix sends `SIGKILL` to the negative process-group identifier, which targets the child and descendants. Windows invokes `taskkill /T /F`. The runtime then kills and waits for the direct child. Cleanup also runs after ordinary completion or provider failure, preventing descendants from outliving a parent that exits early.

4. Empirical method

4.1 Standards used

The report applies two ACM SIGSOFT empirical standards: **Engineering Research**, because RepoTracer is a technological artifact, and **Benchmarking of Software Systems**, because automated workloads compare the artifact with a direct baseline [1–3]. It also uses the ACM Artifact Review and Badging definitions to distinguish same-team repeatability from independent reproducibility and replicability [4]. The NeurIPS paper checklist supplies additional checks for claim scope, limitations, experimental detail, compute reporting, open artifacts, and disclosure of LLM use [5].

These standards are used as reporting criteria, not as a claim of venue compliance or peer review.

4.2 Experimental units and treatments

The final end-to-end comparison used three software-engineering tasks in TypeScript, Python, and Go:

Task	Repository workload	Principal checks
True Myth collection combinators	Add collection combinators across related abstractions	96 feature; 561 regression
sqlfmt DDL formatting	Extend DDL tokenisation and canonical formatting	32 feature; 1,273 regression
Tengo callable isolation	Isolate callable state across VM instances	23 feature; 122 regression

For each pair, the task prompt, base commit, primary model, primary reasoning level, provider path, timeout, and execution policy were held constant. The baseline had no RepoTracer MCP integration. The treatment installed RepoTracer and its routing instructions. The treatment’s complete cost is

$$C_{\text{treatment}} = C_{\text{parent}} + C_{\text{scout}}.$$

For any paired quantity x , the reported percentage change is

$$\Delta_x = 100 \frac{x_{\text{treatment}} - x_{\text{baseline}}}{x_{\text{baseline}}}.$$

A negative cost value indicates lower measured provider cost. A positive wall-time value indicates a slower treatment.

The three-task set is a focused release check. An earlier 30-task, three-repeat matrix stopped after 24 unpaired trials because subscription-account capacity became unavailable. Since the interruption left no matched pairs, those attempts are preserved but excluded from paired effects.

4.3 Quality measures

Quality is measured at three levels.

1. **Exact task reward:** whether the official task verifier accepts the complete patch.
2. **Feature-to-pass checks (F2P):** checks for the requested new behaviour.
3. **Pass-to-pass checks (P2P):** existing regression checks that should remain passing.

Manual review records material implementation differences but does not override executable checks. Scout-only studies use a blind 0–4 answer rubric and validate source citations independently.

4.4 Cost and token accounting

Complete task cost includes every attributed primary-model request and every Scout request. Scout reasoning comparisons use an explicitly labelled *equivalent usage cost*, not a provider invoice. For uncached input I_u , cached input I_c , output O , and published per-million-token rates p_u , p_c , and p_o , the calculation is

$$C = \frac{I_u p_u + I_c p_c + O p_o}{10^6}.$$

Reasoning tokens are a reported subset of output tokens and are not counted twice. Cost comparisons are descriptive because provider rates, cache behaviour, and routing can change.

4.5 Routing studies

Routing evaluation separates first-operation classification from complete-task outcomes.

- The retained v2.1 candidate recheck contains 42 runs: 21 broad requests and 21 local requests.
- A paired holdout contains 48 runs, split evenly between baseline and candidate.
- A six-run boundary pilot contains three local and three Scout cases.
- The rejected v2 diagnostic contains 84 first-operation runs and three repeats per task-arm combination.

First-operation studies establish route selection and citation validity. They do not establish complete-task cost or latency.

4.6 Scout reasoning study

The reasoning study used eight repository questions, three repeats, and three arms: low, medium, and high. All 72 calls used the same Luna model snapshot, subscription app-server path, prompt, output schema, repository snapshot, native tools, concurrency, and inactivity timeout. Only reasoning effort changed. One reviewer scored answers blind to arm on a 0–4 rubric and locked scores before unblinding.

A preregistered hard-task follow-up compared medium and high on two repositories with three repeats per arm. It tested whether high reasoning rescued at least two of three repeats on one task. No early stopping was allowed.

4.7 Timeout fixture

The timeout experiment used five repeats per condition with a 200 ms idle interval. The active fixture emitted valid frames across a total duration longer than the interval. The silent fixture forked a descendant that would create an observable file if it survived. A third fixture exited the parent early while leaving a delayed descendant. Outcomes were binary: active run survived, silent process tree was killed, and early-exit descendants were killed.

The permanent regression test uses the same observable contract at shorter durations. It also verifies token accounting, citation return, app-server arguments, and isolated Codex home selection.

4.8 Execution environment

Local orchestration and release verification for this report ran on an Apple M4 Pro host with Darwin 25.2.0. The Rust workspace declares Rust 1.80 as its minimum supported version. Model inference ran remotely through the Codex subscription app-server path. The final paired task arms used GPT-5.6 Sol at high reasoning; Scout used GPT-5.6 Luna at medium reasoning. The final arm-level wall-time sums were 3,436.527 s for baseline and 4,014.937 s for treatment.

The Scout reasoning study accumulated approximately 4,806.7 s of per-call duration across 72 calls, computed from each arm’s reported mean and 24 trials. This is not elapsed experiment time because calls may overlap.

5. Results

5.1 RQ1: end-to-end quality, cost, and latency

Table 1 reports the focused final paired evaluation.

Measure	Direct baseline	RepoTracer	Difference
Exact task rewards	2/3	2/3	0
Feature checks	147/151	147/151	0
Regression checks	1,952/1,956	1,956/1,956	+4
Mean partial score	0.99796	0.99898	+0.00102
Complete cost	\$20.81148	\$15.04450	-27.71%
Scout component of treatment cost	–	\$0.01272	–
Aggregate wall time	3,436.527 s	4,014.937 s	+16.83%
Valid Scout citations	–	13/13	–
Scout-first routes	–	3/3	–

The exact and feature outcomes tie. RepoTracer preserves four regression checks that the baseline loses on the Python task. Both arms still miss the same four requested canonicalisation fixtures, so neither solves that task exactly.

Cost falls on all three task pairs: 47.91% on the TypeScript task, 21.17% on the Python task, and 4.76% on the Go task. Latency does not follow the same pattern. RepoTracer is 43.99% slower on TypeScript, 21.00% slower on Python, and 11.73% faster on Go. Aggregate latency therefore increases by 16.83%.

Answer to RQ1. In this three-task sample, RepoTracer preserves measured functional quality and reduces complete measured provider cost. The data reject a latency-improvement claim. With only three task pairs, they do not estimate a general effect size.

5.2 Supporting task evidence

Earlier studies provide directional evidence but remain heterogeneous.

Study	Sample	Quality	Cost	Wall time
Immediate routing	3 pairs; one read-only question	6/6 checks in both arms	-28.63% median	+6.65% median
Repeated natural routing	3 pairs; same question	6/6 checks in both arms	-39.20% median	+31.21% median
SWE-bench	1 coding task	Exact regression in both arms	-50.12%	-9.60%
Astropy 13453	1 implementation task	78.75 vs 83.125	-62.68%	-24.54%
Google signup	1 implementation task			
Final release check	3 paired tasks	2/3 exact in both arms	-27.71% total	+16.83% total

These studies differ in task, design, and outcome measure. Pooling them into one average would be misleading.

5.3 RQ2: routing quality

Ownership-first routing v2.1 passed all 42 candidate recheck runs: 21 of 21 broad prompts called Scout and 21 of 21 local prompts stayed local. All 102 returned citations were valid and no run timed out.

The 48-run paired holdout tied perfectly. Each arm passed 24 of 24 cases, with 12 of 12 broad prompts routed to Scout and 12 of 12 local prompts kept local. Candidate cost was 11.30% higher and wall time 6.03% higher in this first-operation diagnostic. The protocol treats those differences as request noise rather than complete-task effects.

The six-run boundary pilot passed all cases, returned 13 valid citations from 13, and recorded no timeout. One complete timeout task then passed both verifiers in each arm; the candidate used one Scout call, returned five valid citations, cost 44.69% less, and finished 6.90% faster. This single pair confirms operation, not a general effect.

Answer to RQ2. The retained policy separates broad and local requests on the measured recheck and holdout sets. The rejection of v2 demonstrates that the boundary is sensitive to wording and requires held-out local cases.

5.4 RQ3: Scout reasoning level

Reasoning	Blind quality	Valid citations	Wall time, median / p95	Equivalent cost
Low	3.75/4; 18/24 perfect	100%	34.590 / 49.123 s	\$0.042642
Medium	4.00/4; 24/24 perfect	100%	58.148 / 72.749 s	\$0.064867
High	4.00/4; 24/24 perfect	100%	138.646 / 185.778 s	\$0.128335

Medium beats low in six paired grades and never loses. The task-cluster bootstrap 95% interval for the medium-minus-low mean score is [0.083, 0.458]. High ties medium on every grade, uses 6.33 times the reasoning-output tokens, and takes 2.38 times the arm-level median wall time.

Low succeeds on every exact lookup but produces six completeness defects across cross-file tasks. A hybrid low-for-exact policy was not retained because the same dataset both suggested and measured that split.

In the preregistered hard-task follow-up, medium passes three of six official verifiers and high passes four of six. High gains one recursive-delegation pass but does not rescue at least two of three repeats on either task. Boa passes remain tied at one of three with opposite repeat winners. High is slower at the median on both tasks. The preregistered promotion gate therefore fails.

Answer to RQ3. Medium is the least costly measured setting that achieves perfect blind grades across the eight-task Scout study. High has no demonstrated production advantage under the preregistered gate.

5.5 RQ4: inactivity timeout and process-tree cleanup

Runtime	Active run survived	Silent tree killed	Early-exit descendants killed
Fixed wall-clock baseline	0/5	5/5	5/5
Activity-based candidate	5/5	5/5	5/5

The active medians were 215 ms for baseline and 444 ms for candidate. Silent medians were 214 ms and 215 ms, respectively. The candidate remains alive for more than twice the 200 ms idle interval because valid frames reset the deadline. Silence still terminates at approximately one idle interval. The descendant checks show that cleanup targets the process tree rather than only the direct child.

Answer to RQ4. The fixture separates duration from inactivity: continuing output extends the run in every repeat, while silence and early parent exit leave no surviving descendant in any repeat.

6. Interpretation

The evidence supports a narrow mechanism. Repository retrieval can move from an expensive primary model to a cheaper Scout without reducing measured quality when routing is selective and the handoff is evidence-bounded. The final task sample shows a complete cost reduction, but the latency cost is real and variable. RepoTracer should therefore be understood as a cost-and-context tool, not a general speed optimisation.

The Scout cost itself is small in the final comparison: \$0.01272 of \$15.04450. Most savings come from reducing primary-model work after the handoff. This also explains why a Scout can add wall time while reducing cost. The parent waits for a serial investigation, then performs fewer expensive turns.

Routing is part of the treatment. Always delegating would erase local-task precision and add needless latency. Never delegating leaves the primary model to reconstruct broad ownership maps. The retained policy uses explicit ownership and exhaustion criteria, then includes a one-lookup escalation rule for ambiguous cases.

The inactivity deadline addresses a different failure mode. It does not improve model quality or route selection. It prevents a healthy stream from being killed by elapsed wall time and prevents a silent process from living indefinitely. Treating this as a total timeout would recreate the original defect.

7. Threats to validity

7.1 Construct validity

Functional quality. Official checks are stronger than textual similarity, but they only cover encoded behaviours. The Python task shows the distinction: both arms pass most checks and still miss four requested cases.

Cost. Complete cost includes attributed parent and Scout requests, which is the relevant user-facing measure. Equivalent Scout cost in the reasoning study uses documented rates rather than invoices. Provider pricing and cache policy can change.

Latency. Wall time includes provider queueing, model inference, tool calls, and local orchestration. It measures user-visible duration but cannot identify which component caused a difference.

Citation validity. Path and line validation proves existence and containment, not semantic entailment. Blind review and task checks provide separate evidence of usefulness.

Routing quality. First-operation labels measure the intended policy, not full task success. Complete-task studies remain necessary.

7.2 Internal validity

Paired tasks hold prompt, repository commit, model settings, and limits constant. Provider-side nondeterminism, queueing, and cache state remain uncontrolled. Randomised execution reduces but does not eliminate time-order effects.

The reasoning study changes only declared reasoning effort and blinds the reviewer. It uses one reviewer, so there is no inter-rater reliability estimate. The hard-task follow-up applies a preregistered gate, but two tasks and three repeats cannot estimate general pass rates.

The interrupted 30-task matrix is excluded from paired claims. Failed provider attempts remain documented rather than silently replaced. One release-grade tool-impact experiment had no CodeGraph index in either repository and produced no useful result in either arm; it supports the timeout result only and is not used to claim repository-tool benefit.

7.3 External validity

The final comparison contains three tasks and three languages. The Scout reasoning study uses eight questions from one repository snapshot. The hard-task study uses two repositories. Results may change with repository size, language, model snapshot, provider, task type, or integration policy.

No result supports a universal token reduction, latency improvement, or extension of every subscription quota. A larger preregistered matrix of 30 independent tasks with three randomised repeats per arm remains the launch-grade evidence target.

7.4 Conclusion validity

The report gives descriptive paired effects for the three final tasks and does not run a significance test on $n = 3$. A confidence interval from three heterogeneous tasks would imply precision the sample does not contain. The 72-trial reasoning analysis reports task-cluster bootstrap intervals because repeats are nested within tasks.

Multiple studies informed policy changes, creating development-set exposure. Retained claims rely on later holdouts or are labelled exploratory. The same-task repeated routing studies are not treated as independent repository samples.

8. Reproducibility and artifact record

The repository contains:

- versioned task prompts and expected-path metadata under `benchmarks/tasks/`;
- runners and analysis programs under `benchmarks/runners/`;
- study protocols and sanitised summaries under `benchmarks/results/runs/`;
- schema definitions for manifests, plans, trials, and reviews;
- checksums for retained public artifacts;
- unit and behavioural checks for citation validation, isolation, setup, routing configuration, and process cleanup;

- a release verifier that builds the binary in a temporary home, registers MCP, performs a live `repo_scout` call, validates citations against source files, and tests uninstall.

Raw model sessions, provider metadata, stdout, stderr, and commands are stored in ignored `private/` directories. This prevents credential and personal-session disclosure, but it means the public repository alone is not a complete ACM-style reproduction package. The software and sanitised evidence may be repeatable by the same team with provider access. Independent reproducibility has not been demonstrated.

A faithful independent attempt should record:

1. repository URL and immutable base commit for every task;
2. exact task prompt, arm, repeat, and randomisation order;
3. primary and Scout model identifiers, reasoning levels, service tiers, and provider path;
4. all timeout, turn, tool, output, and citation limits;
5. complete parent-plus-Scout request, token, cost, and wall-time accounting;
6. exact verifier commands and per-check outcomes;
7. unsuccessful trials and exclusion reasons;
8. raw artifacts in a credential-scrubbed archival package with a stable identifier.

9. Standards conformance assessment

9.1 ACM SIGSOFT Benchmarking standard

Criterion	Status	Evidence or deviation
Defines quality, metrics, measurement, and workload	Met	Sections 1 and 4 define functional checks, cost, time, routing, citations, and tasks
Justifies benchmark design	Partly met	Paired real-repository tasks and official checks are relevant; representativeness is not established
Describes setup and workload for replication	Partly met	Protocols, commits, prompts, models, and runners are recorded; provider access and private traces limit independent execution
Allows configurations to compete without artificial limits	Met for paired arms	The intended treatment difference is RepoTracer integration; shared limits are held constant
Uses sufficient repetitions and duration	Mixed	Routing and reasoning studies repeat; the final three-task comparison has one retained pair per task
Discusses construct validity	Met	Section 7.1
Supplies datasets and analysis scripts	Partly met	Sanitised summaries and scripts are public; credential-bearing raw traces are withheld
Reports execution problems transparently	Met	Interrupted, invalidated, unavailable-provider, and inconclusive runs are recorded
Independent replication	Not met	No independent team has reproduced the results

9.2 ACM SIGSOFT Engineering Research standard

Criterion	Status	Evidence or deviation
Describes the artifact and workflow	Met	Section 3 and repository source
Justifies relevance	Met	Section 1 identifies primary-model search cost and context pressure
Evaluates strengths, weaknesses, and limits	Met	Sections 5–7
Names the empirical method	Met	Paired software-system benchmarking
Discusses alternatives	Met	Direct baseline, rejected routing v2, low/medium/high Scout reasoning
Compares with an alternative	Met	Direct parent-agent baseline
Makes assumptions explicit	Met	Sections 4 and 7
Provides source and inputs	Partly met	Source and sanitised inputs are present; raw provider sessions are private
Industry-relevant context	Partly met	Real open-source repositories and production subscription path; no professional-user study

9.3 Reproducibility and responsible-reporting checklist

Item	Status	Note
Claims match evidence scope	Yes	Abstract and conclusion limit claims to measured workloads
Dedicated limitations discussion	Yes	Section 7
Experimental details	Yes	Section 4 and versioned protocols
Statistical uncertainty	Partial	Cluster bootstrap for reasoning; no inferential claim for the three-task final sample
Open code and data	Partial	Code and sanitised aggregates are available; raw sessions are private
Compute resources	Partial	Local host, model path, and measured durations are reported; provider hardware is unknown
LLM use disclosed	Yes	Both treatment and baseline are model-backed; model roles and settings are named
Human participants	Not applicable	No human-subject experiment was conducted; one reviewer graded model output
Broader impacts and safeguards	Yes	Section 10
Independent validation	No	No external reproduction or replication has occurred

Under ACM terminology, this report claims neither **Artifacts Evaluated**, **Artifacts Available**, nor **Results Validated**. A mutable source repository without independent audit or archival identifier is not sufficient for those labels [4].

10. Security, ethics, and broader impacts

RepoTracer reduces the amount of broad source code sent through an expensive primary model by delegating read-only search to another provider-backed model. It does not eliminate source disclosure to model providers. Users must assess repository confidentiality and provider terms before use.

The main safeguards are least-privilege execution, repository-root enforcement, symlink-escape rejection, no shell interpolation in repository tools, bounded tool output, isolated child configuration, no inherited MCP servers, and validation of source citations before handoff. These controls reduce accidental capability expansion; they do not make model output trustworthy.

A precise repository map can accelerate both maintenance and malicious analysis. RepoTracer has no autonomous write capability, but a parent coding agent may act on its output. The parent must review cited code, preserve normal authorisation boundaries, and run project verification before accepting changes.

The experiments use public software repositories and provider services. No human subjects, crowdsourced workers, or personal datasets are part of the benchmark. Private authentication and session data are intentionally excluded from public artifacts.

11. Conclusions

RepoTracer 1.0 implements a bounded division of labour: the primary coding model owns changes, while a smaller read-only Scout maps broad repository surfaces and returns validated evidence. Ownership-first routing avoids delegating a task merely because its file path is unknown. A resettable inactivity deadline lets active subscription Scouts run for as long as they continue producing meaningful stream activity, yet kills silent process trees.

The strongest end-to-end evidence is a three-task paired release check. It shows equal exact and feature outcomes, four additional preserved regression checks, and 27.71% lower complete measured cost for RepoTracer, alongside 16.83% higher aggregate wall time. This supports a cost-and-context claim for the measured tasks, not a speed claim or general effect estimate.

The 72-trial blind reasoning study supports medium reasoning over low and high for the current Scout workload. Routing v2.1 passes the measured broad/local recheck and holdout sets. Five repeated timeout fixtures establish the intended activity and cleanup behaviour.

The remaining scientific gap is breadth. A preregistered multi-repository matrix with independent task sampling, repeated paired runs, multiple blind reviewers, provider-version capture, and an archival reproduction package is required before making population-level claims.

References

- [1] Paul Ralph et al. “Empirical Standards for Software Engineering Research.” arXiv:2010.03525, 2020. <https://arxiv.org/abs/2010.03525>.
- [2] ACM SIGSOFT. “Benchmarking (of Software Systems).” *Empirical Standards for Software Engineering Research*. Accessed 30 August 2026. <https://github.com/acmsigsoft/EmpiricalStandards/blob/master/docs/standards/Benchmarking.md>.

- [3] ACM SIGSOFT. “Engineering Research (AKA Design Science).” *Empirical Standards for Software Engineering Research*. Accessed 30 August 2026. <https://github.com/acmsigsoft/EmpiricalStandards/blob/master/docs/standards/EngineeringResearch.md>.
- [4] Association for Computing Machinery. “Artifact Review and Badging, Version 1.1.” 24 August 2020. <https://www.acm.org/publications/policies/artifact-review-and-badging-current>.
- [5] NeurIPS. “Paper Checklist Guidelines.” Accessed 30 August 2026. <https://neurips.cc/public/guides/PaperChecklist>.
- [6] Jóakim von Kistowski, Jeremy A. Arnold, Karl Huppler, Klaus-Dieter Lange, John L. Henning, and Paul Cao. “How to Build a Benchmark.” In *Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering*, 2015. <https://doi.org/10.1145/2668930.2688819>.
- [7] Samuel Kounev, Klaus-Dieter Lange, and Jóakim von Kistowski. *Systems Benchmarking for Scientists and Engineers*. Springer, 2020. <https://doi.org/10.1007/978-3-030-41705-5>.
- [8] RepoTracer Contributors. “RepoTracer source, benchmark protocols, and sanitised result summaries.” Version 1.0.0, 2026. <https://github.com/repotracer/repotracer>.